

Les chaînes de caractères (*String*) et les caractères

Les exercices marqués d'une étoile (*) ne seront pas corrigés en classe. Ils font néanmoins partie de la matière.

- 1) Ecrivez une fonction qui détermine si un caractère est alphabétique minuscule. Ne vous occupez pas des diacritiques (accents).
- 2) Ecrivez une fonction qui renvoie la majuscule d'un caractère donné. Que renvoyez-vous si le caractère n'est pas une minuscule ?
- 3) Ecrivez une fonction qui détermine si un string est formé exclusivement de minuscules. Utilisez votre fonction de la question 1.
- 4) Ecrivez une fonction qui renvoie une copie en majuscule d'un string donné.
- 5) (*) Ecrivez une fonction qui compte le nombre d'apparitions d'un caractère donné dans un string. Par exemple, le caractère 'o' apparaît 2 fois dans le mot "Onomatopée" car nous distinguons minuscules et majuscules.
- 6) Ecrivez une fonction qui renvoie une copie renversée d'un string. Par exemple le string "hello" donne "olleh".
- 7) Ecrivez une fonction qui détermine si un string forme un palindrome (il est égal à sa copie renversée). Par exemple *radar* forme un palindrome. Est-ce nécessaire de le renverser pour le déterminer ?
- 8) (*) Refaites l'exercice précédent pour un string pouvant être formé de plusieurs mots. Dans ce cas, il convient d'ignorer les espaces et les différences entre minuscules et majuscules. Par exemple : "Esopé reste ici et se repose". Pour permettre à l'utilisateur d'entrer une phrase entière, utilisez un `Scanner` avec la méthode `nextLine()`.
- 9) (*) Ecrivez une fonction qui renvoie l'indice de la première apparition d'un caractère dans un string. Que renvoyez-vous si le caractère n'apparaît pas dans le string ?
- 10) Comme vu dans les slides, vous ne pouvez pas utiliser l'opérateur `==` pour déterminer l'identité de deux chaînes de caractères. Par contre vous pouvez bien sûr l'utiliser pour comparer des caractères. Ecrivez, sans utiliser la méthode `compareTo()`, une fonction : `boolean sontEgaux(String s1, String s2)`
- 11) (*) Ecrivez une fonction `boolean inclus(String ssch, String ch)` qui renvoie **true** si et seulement si tous les caractères de *ssch* se trouvent, dans le même ordre, dans *ch*, mais pas nécessairement de façon contiguë. Par exemple :
`inclus("mai", "machin")` → `true`
`inclus("mai", "misérable")` → `false`
- 12) Comme vu dans les slides, en Java, les opérateurs de comparaison (`<`, `>`, ...) ne sont pas définis sur les chaînes de caractères (mais bien sur les caractères eux-mêmes). Ecrivez, sans utiliser la méthode `compareTo()`, la fonction `int compare(String ch1, String ch2)` qui renvoie un résultat négatif (quelconque) si *ch1* < *ch2*, nul si *ch1* == *ch2* et positif si *ch1* > *ch2*.

-
- 13)(*) Ecrivez une fonction qui renvoie une copie d'un string donné en mettant en majuscule la première lettre de chaque mot du string. On suppose (sans le vérifier) que le string est composé uniquement de caractères alphabétiques et d'espaces, ces derniers séparant les mots. Il n'y a pas de ponctuation, par exemple. Pour permettre à l'utilisateur d'entrer une phrase entière, utilisez un `Scanner` avec la méthode `nextLine()`.
- 14) Ecrivez une fonction `intPositifToString()` qui reçoit un entier **positif ou nul** et renvoie un string contenant la représentation décimale de cet entier. Par exemple l'entier `123` donne le **String** (texte) `"123"`. Votre fonction doit déclencher une exception si l'entier est négatif. Ecrivez une autre fonction `intToString()` qui peut recevoir tout entier positif ou négatif. Utilisez la première fonction.
- 15)(*) Ecrivez une fonction boolean `sousChaine(String ssch, String ch)` qui renvoie **true** si et seulement si `ssch` est une sous-chaîne de `ch` (`ssch` apparaît comme une partie de `ch`). Par exemple :
- ```
sousChaine("mai", "primaire") → true
sousChaine("mai", "machin") → false
```
- 16) Ecrivez un programme qui affiche les fréquences d'apparitions des caractères alphabétiques d'un texte entré par l'utilisateur. Les autres caractères sont ignorés. Majuscules et minuscules sont comptabilisées conjointement. Par exemple, le texte « La vie est belle » donnera l'affichage :  
A: 1, B: 1, E: 4, I: 1, L: 3, S: 1, T: 1, V: 1. Pour permettre à l'utilisateur d'entrer une phrase entière, utilisez un `Scanner` avec la méthode `nextLine()`. Utilisez un tableau (*Array*) pour mémoriser les nombres d'apparitions des différents caractères.